Lesson 06 Demo 04

Creating a React Application Demonstrating Retrieval of Data

Objective: To develop a React application that demonstrates retrieval of data asynchronously to enhance performance and improve user experience

Tools required: Node Terminal, React app, and Visual Studio Code

Prerequisites: Knowledge of creating a React app and understanding of the folder structure

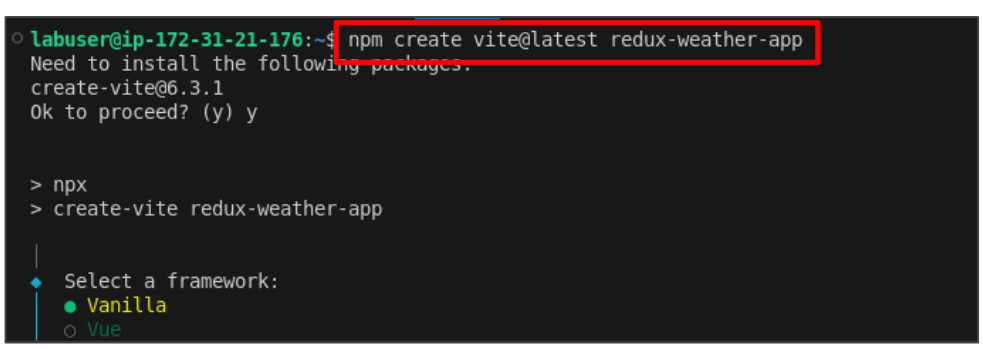
Steps to be followed:

1. Create a new React app
2. Create a new file called reducers.js
3. Create a new file called types.js
4. Create a new file called actions.js
5. Get the API Key
6. Create a new file called Weather.js
7. Create a new file called WeatherForm.js
8. Open the existing file App.js in the src folder
9. Create a new file called index.js
10. Create a new file called .env
11. Run the app and view it in the browser

Step 1: Create a new React app

1.1 Open your terminal and run the following command:

npm create vite@latest redux-weather-app



1.2 Select **React** as **framework** and **JavaScript** as **variant**

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

◇ Select a framework:
  React

◆ Select a variant:
  ○ TypeScript
  ○ TypeScript + SWC
  ● JavaScript
  ○ JavaScript + SWC
  ○ React Router v7 ↗
```

1.3 Move to the redux-weather-app directory by executing the following command:
cd redux-weather-app

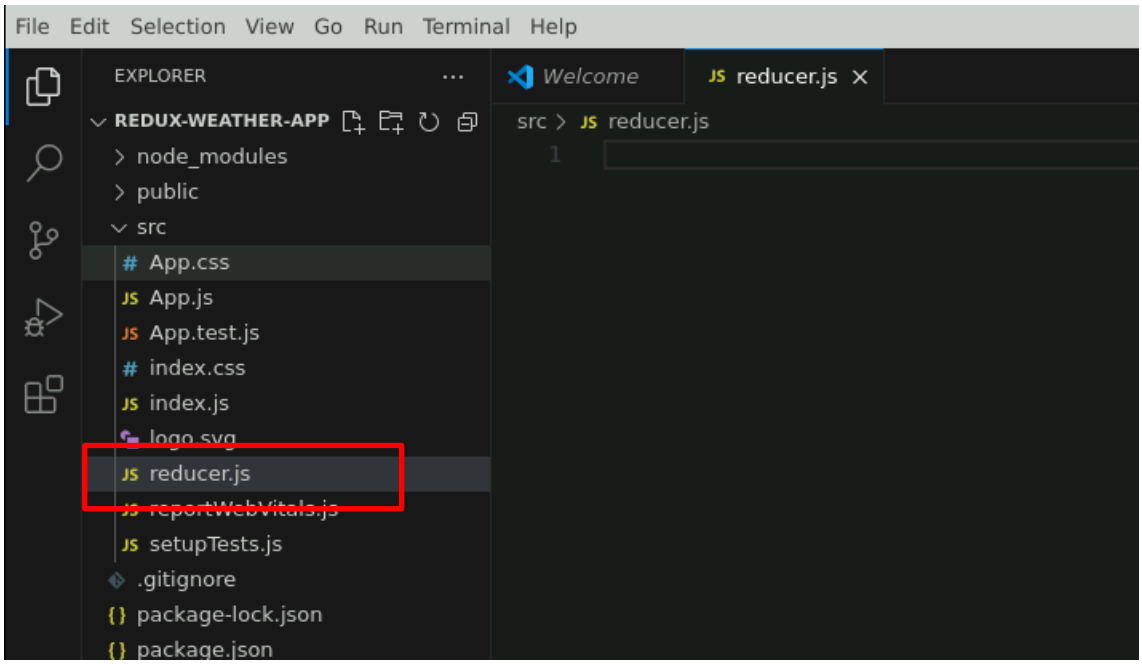
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

● labuser@ip-172-31-30-250:~$ cd redux-weather-app
○ labuser@ip-172-31-30-250:~/redux-weather-app$
```

1.4 Install the necessary dependencies by running the following command:
npm install react redux react-redux redux-thunk axios

```
labuser@ip-172-31-30-250:~/redux-weather-app$ npm install react redux react-redux redux-thunk axios

added 8 packages, and audited 1351 packages in 6s

268 packages are looking for funding
  run `npm fund` for details

8 vulnerabilities (2 moderate, 6 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

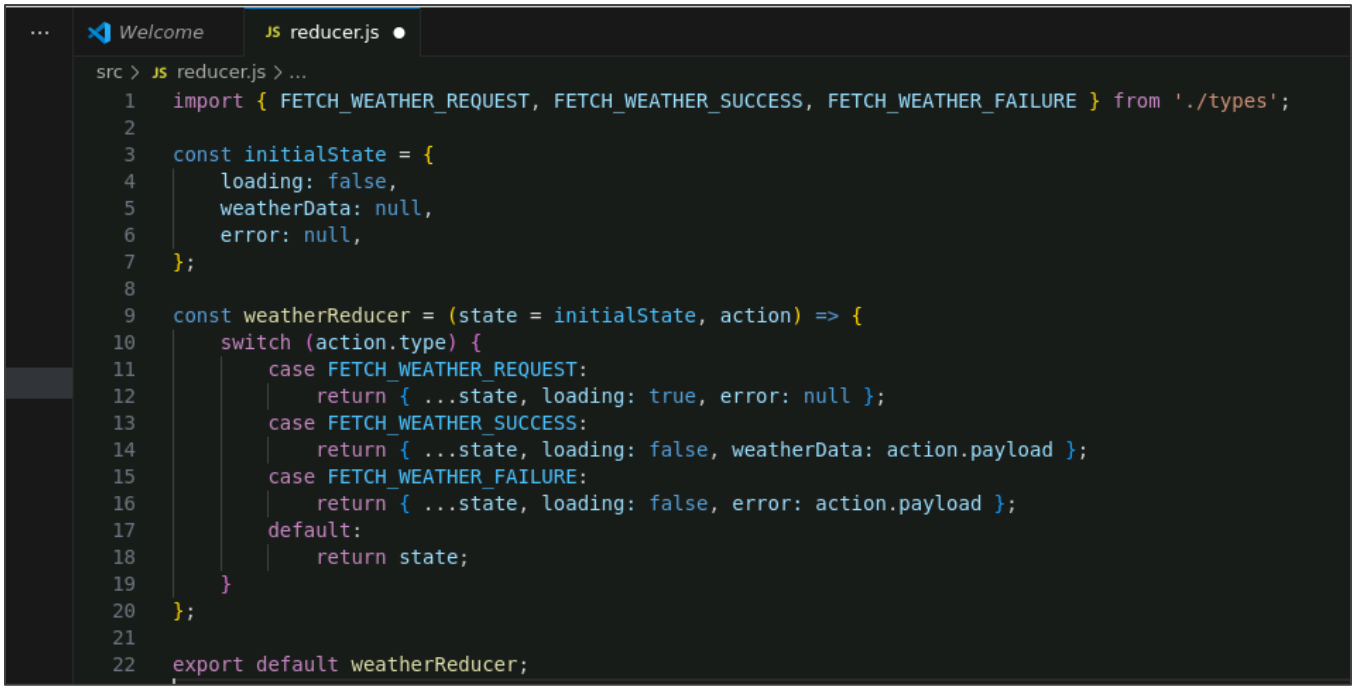
Step 2: Create a new file called reducers.js

2.1 Open VS Code, navigate to the project folder, and create a **reducers.js** file in the **src** directory



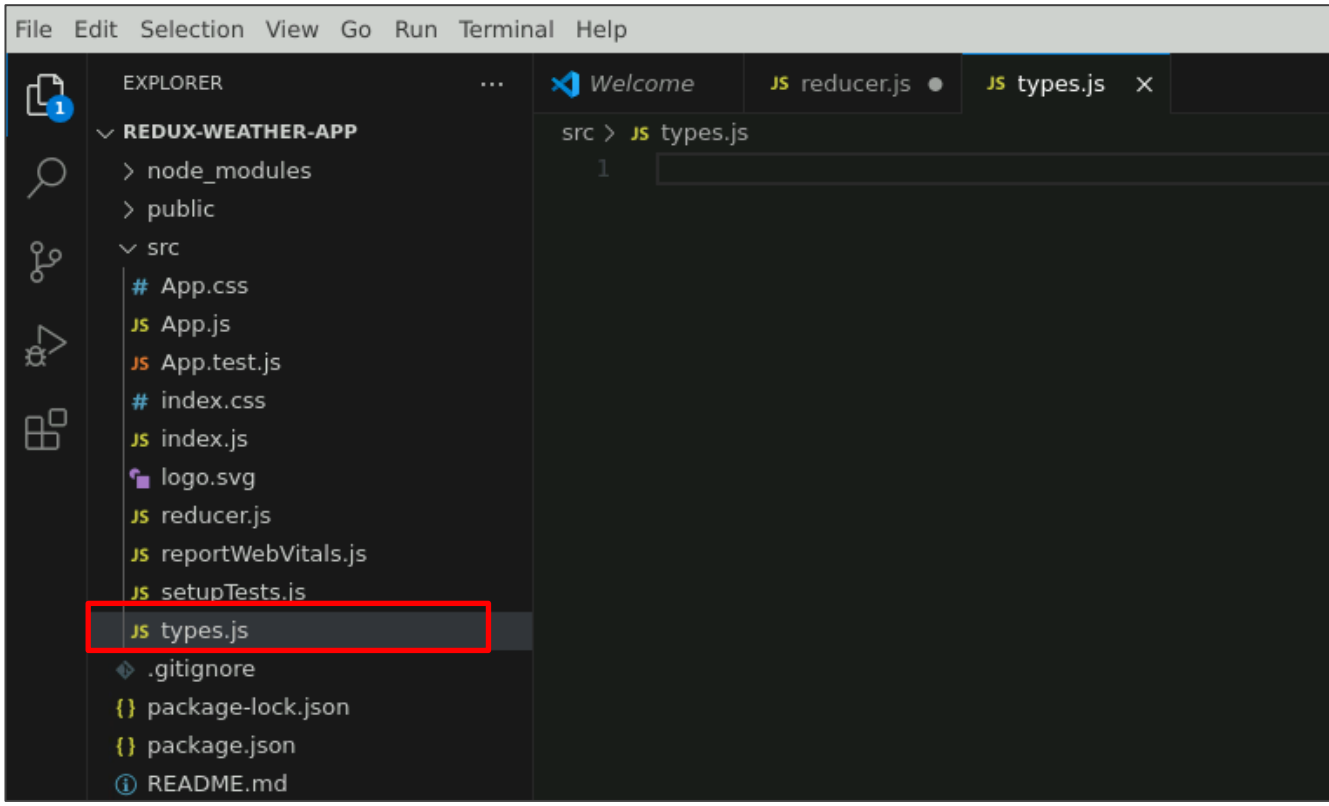
2.2 Enter the following code under **reducers.js**:

```
import { FETCH_WEATHER_REQUEST, FETCH_WEATHER_SUCCESS,
FETCH_WEATHER_FAILURE } from './types';

const initialState = {
  loading: false,
  weatherData: null,
  error: null,
};

const weatherReducer = (state = initialState, action) => {
  switch (action.type) {
    case FETCH_WEATHER_REQUEST:
      return { ...state, loading: true, error: null };
    case FETCH_WEATHER_SUCCESS:
      return { ...state, loading: false, weatherData: action.payload };
    case FETCH_WEATHER_FAILURE:
      return { ...state, loading: false, error: action.payload };
    default:
      return state;
  }
};

export default weatherReducer;
```



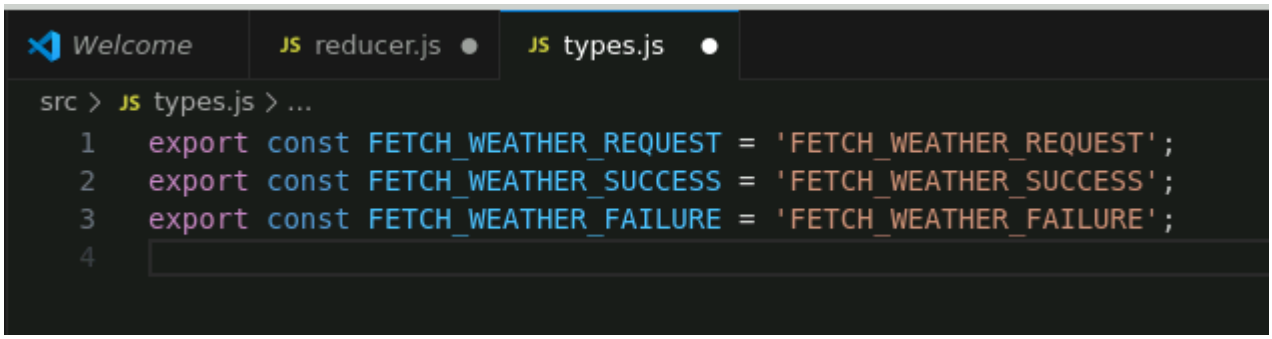
Step 3: Create a new file called types.js

3.1 In the **src** directory, create a new file named **types.js**



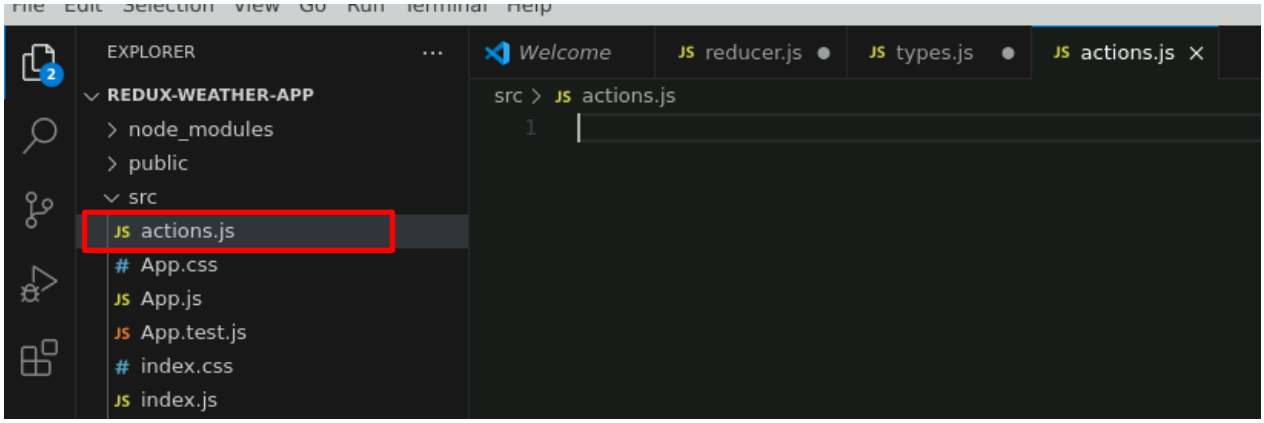
3.2 Add the following code under the **types.js** file:

export const FETCH_WEATHER_REQUEST = 'FETCH_WEATHER_REQUEST';
export const FETCH_WEATHER_SUCCESS = 'FETCH_WEATHER_SUCCESS';
export const FETCH_WEATHER_FAILURE = 'FETCH_WEATHER_FAILURE';



Step 4: Create a new file called actions.js

4.1 In the **src** directory, create a new file named **actions.js**



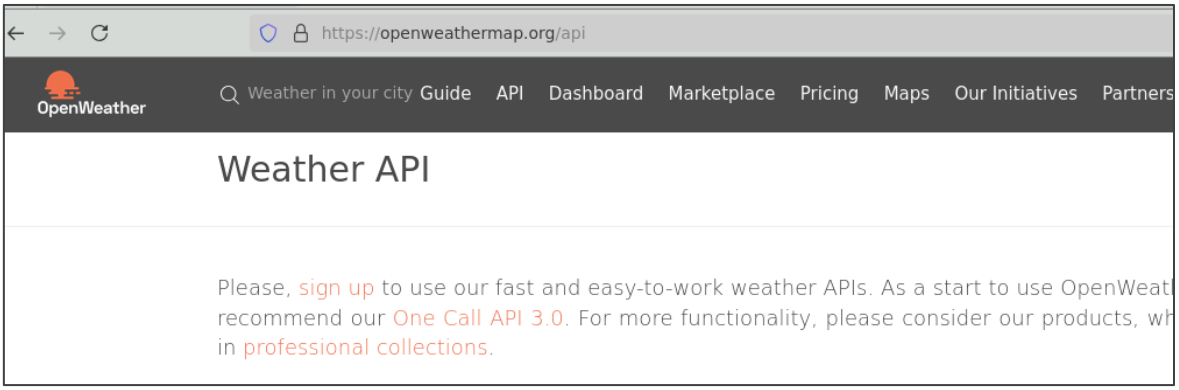
4.2 Add the following code under **actions.js:**

```
import axios from 'axios';
import { FETCH_WEATHER_REQUEST, FETCH_WEATHER_SUCCESS,
    FETCH_WEATHER_FAILURE } from './types';

export const fetchWeather = (city) => {
  return async (dispatch) => {
    dispatch({ type: FETCH_WEATHER_REQUEST });
    try {
      const response = await
    axios.get(`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${pr
    ocess.env.REACT_APP_API_KEY}`);
      dispatch({ type: FETCH_WEATHER_SUCCESS, payload: response.data });
    } catch (error) {
      dispatch({ type: FETCH_WEATHER_FAILURE, payload: error.message });
    }
  };
  };
```

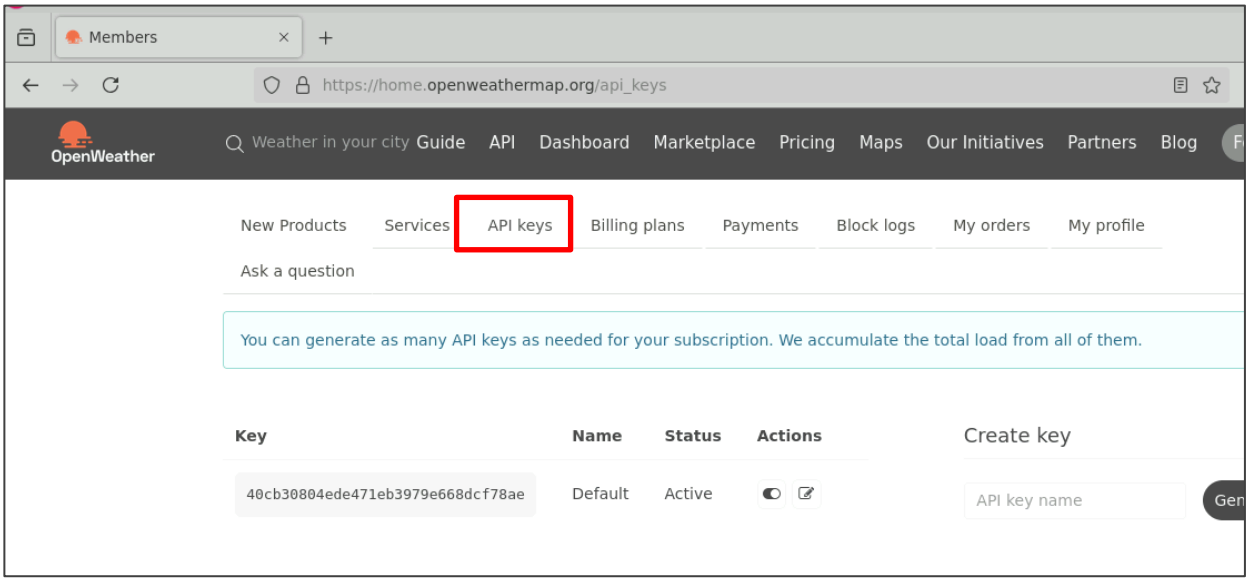
```
Welcome   reducer.js   types.js   actions.js
src > actions.js > ...
1  import axios from 'axios';
2  import { FETCH_WEATHER_REQUEST, FETCH_WEATHER_SUCCESS, FETCH_WEATHER_FAILURE } from './types
3
4  export const fetchWeather = (city) => {
5      return async (dispatch) => {
6          dispatch({ type: FETCH_WEATHER_REQUEST });
7          try {
8              const response = await axios.get(`https://api.openweathermap.org/data/2.5/weathe
9              dispatch({ type: FETCH_WEATHER_SUCCESS, payload: response.data });
10         } catch (error) {
11             dispatch({ type: FETCH_WEATHER_FAILURE, payload: error.message });
12         }
13     };
14 };
```

Step 5: Get the API key

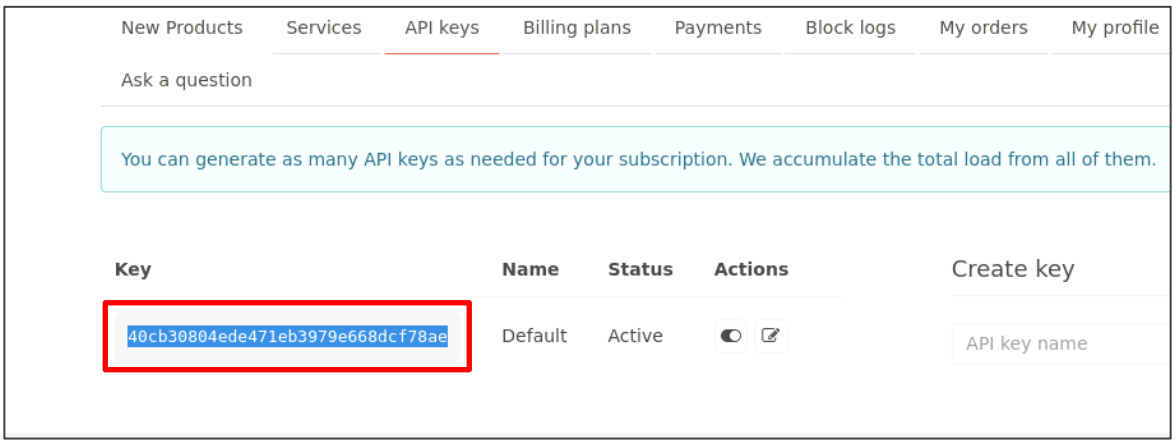
5.1 Navigate to **https://api.openweathermap.org** and sign up



5.2 Click on **API keys** on the home page



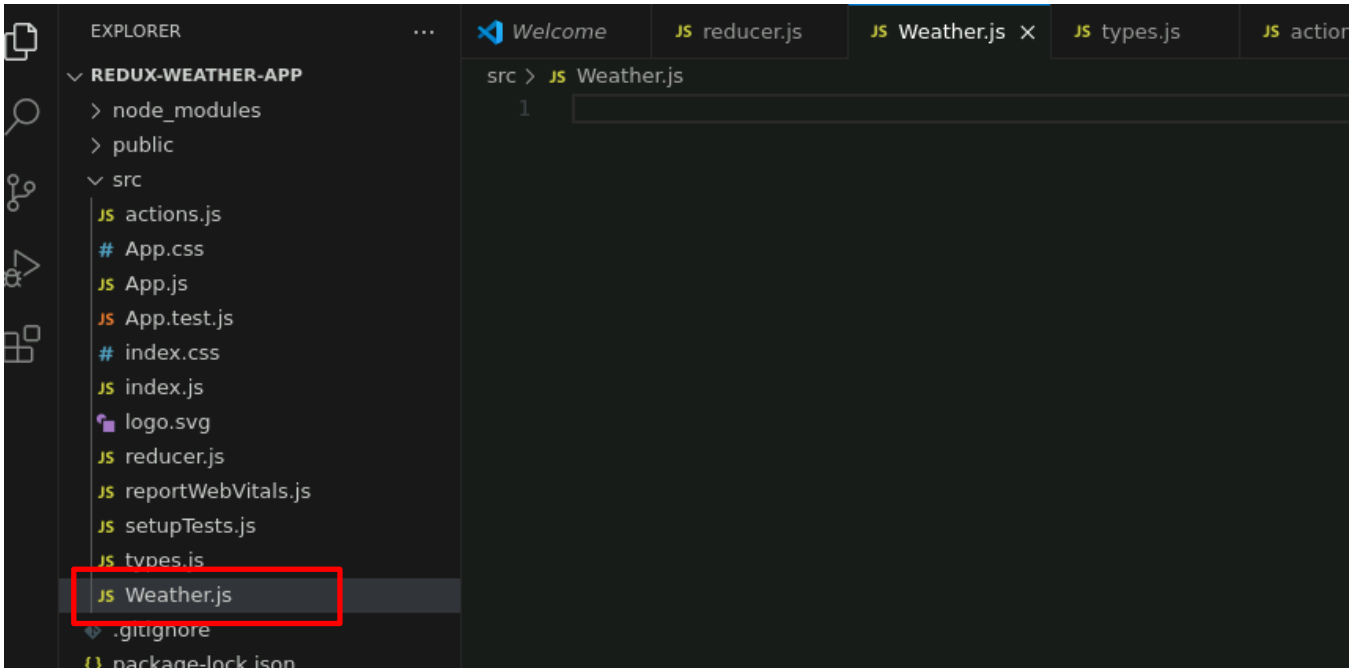
5.3 Copy the key for further use



This will contain the action creators that will dispatch the actions to the reducer.

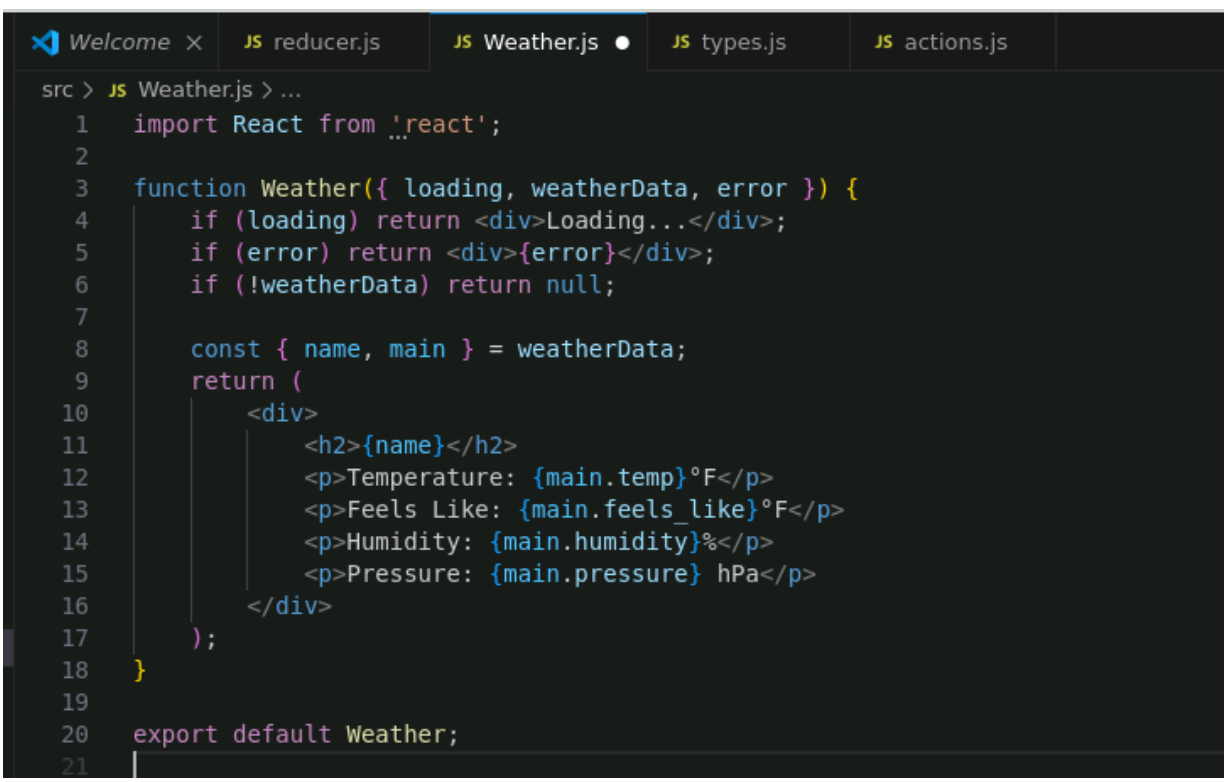
Step 6: Create a new file called Weather.js

6.1 In the **src** directory, create a new file called **Weather.js**



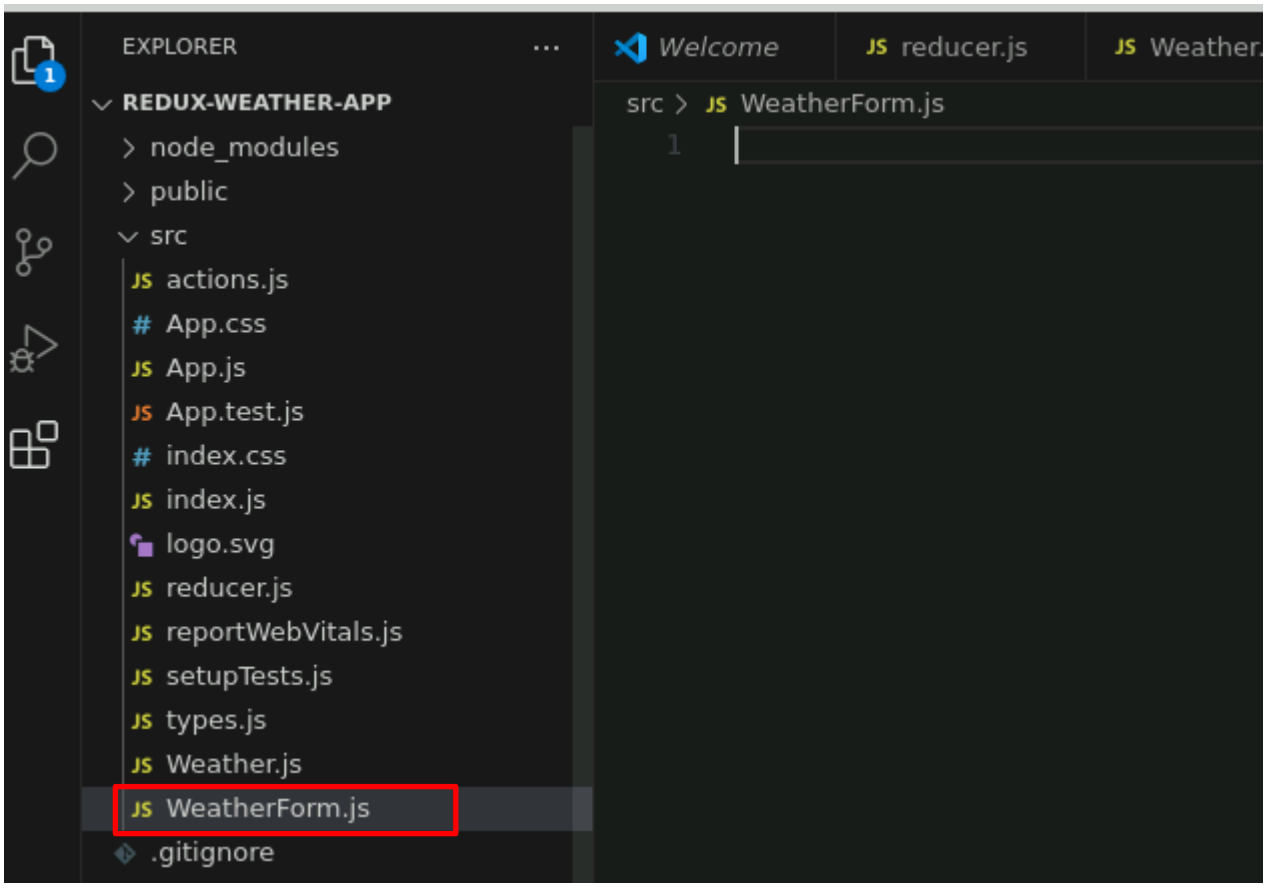
6.2 Add the following code inside the **Weather.js** file:

```
import React from 'react';

function Weather({ loading, weatherData, error }) {
  if (loading) return <div>Loading...</div>;
  if (error) return <div>{error}</div>;
  if (!weatherData) return null;

  const { name, main } = weatherData;
  return (
    <div>
      <h2>{name}</h2>
      <p>Temperature: {main.temp}°F</p>
      <p>Feels Like: {main.feels_like}°F</p>
      <p>Humidity: {main.humidity}%</p>
      <p>Pressure: {main.pressure} hPa</p>
    </div>
  );
}

  export default Weather;
```



Step 7: Create a new file called WeatherForm.js

5.1 In the **src** directory, create a new file named **WeatherForm.js**



5.2 Enter the following code inside the file:

```
import React, { useState } from 'react';

function WeatherForm({ onSubmit }) {
  const [city, setCity] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit(city);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={city} onChange={(e) => setCity(e.target.value)}
   placeholder="Enter city" />
```

```
        <button type="submit">Get Weather</button>
      </form>
    );
  }

    export default WeatherForm;
```

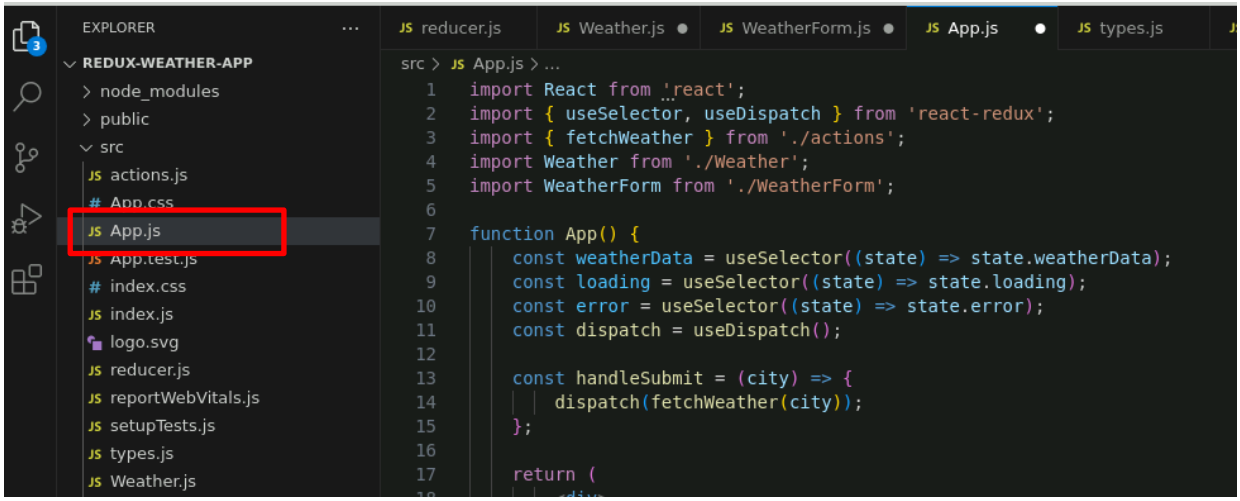
```
Welcome | reducer.js | Weather.js ● | WeatherForm.js ● | types.js | actions
src > WeatherForm.js > ...
1   import React, { useState } from 'react';
2
3   function WeatherForm({ onSubmit }) {
4       const [city, setCity] = useState('');
5
6       const handleSubmit = (e) => {
7           e.preventDefault();
8           onSubmit(city);
9       };
10
11      return (
12          <form onSubmit={handleSubmit}>
13              <input type="text" value={city} onChange={(e) => setCity(e.target.v
14              <button type="submit">Get Weather</button>
15          </form>
16      );
17  }
18
19  export default WeatherForm;
```

This will be our presentational component that will display the form to enter a city and submit it to fetch the weather data.

Step 8: Open the existing file App.js in the src folder

8.1 In the **src** directory, create a new file named **App.js** and enter the following code:

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { fetchWeather } from './actions';
import Weather from './Weather';
import WeatherForm from './WeatherForm';

function App() {
  const weatherData = useSelector((state) => state.weatherData);
  const loading = useSelector((state) => state.loading);
  const error = useSelector((state) => state.error);
  const dispatch = useDispatch();

  const handleSubmit = (city) => {
    dispatch(fetchWeather(city));
  };

  return (
    <div>
      <h1>Weather App</h1>
      <WeatherForm onSubmit={handleSubmit} />
      <Weather loading={loading} weatherData={weatherData} error={error} />
    </div>
  );
}

  export default App;
```



Step 9: Create a new file called index.js

9.1 In the **src** directory, open the **index.js** file and enter the following code inside the file:

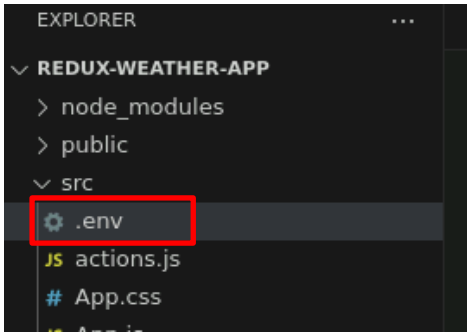
```
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import { createStore, applyMiddleware } from 'redux';
import thunk from 'redux-thunk';
import weatherReducer from './reducers';
import App from './App';

const store = createStore(weatherReducer, applyMiddleware(thunk));

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

```
JS reducer.js   JS Weather.js ●   JS WeatherForm.js ●   JS App.js   JS index.js
src > JS index.js > ...
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3  import { Provider } from 'react-redux';
4  import { createStore, applyMiddleware } from 'redux';
5  import thunk from 'redux-thunk';
6  import weatherReducer from './reducers';
7  import App from './App';
8
9  const store = createStore(weatherReducer, applyMiddleware(thunk));
10
11 ReactDOM.render(
12     <Provider store={store}>
13         <App />
14     </Provider>,
15     document.getElementById('root')
16 );
17
```

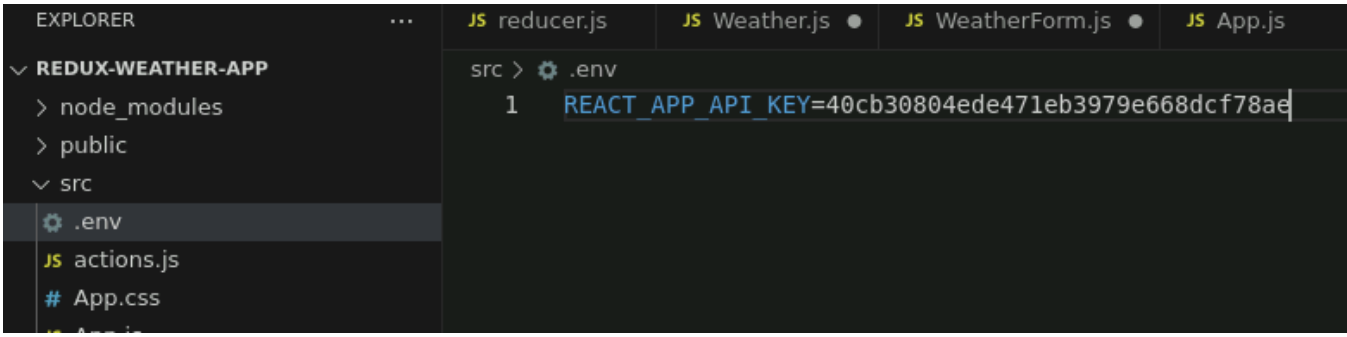
This will be the entry point of our application.

Step 10: Create a new file called .env

10.1 In the **src** directory, create a new file named **.env**

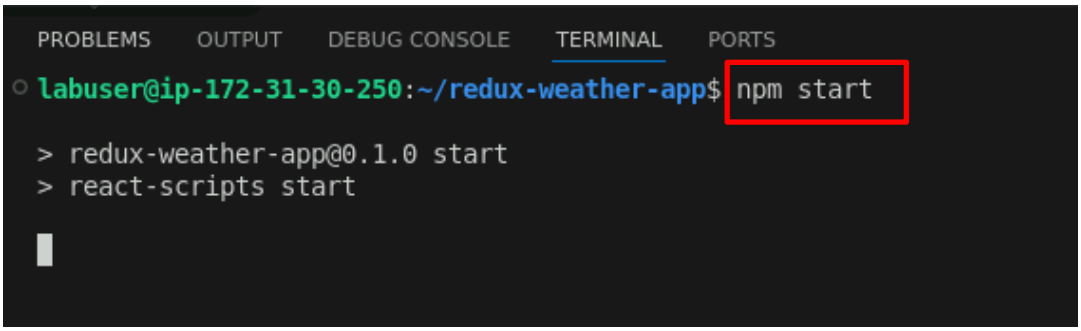


10.2 Add the following line in the **.env** file, add previously copied API key:
REACT_APP_API_KEY=YOUR_API_KEY_HERE



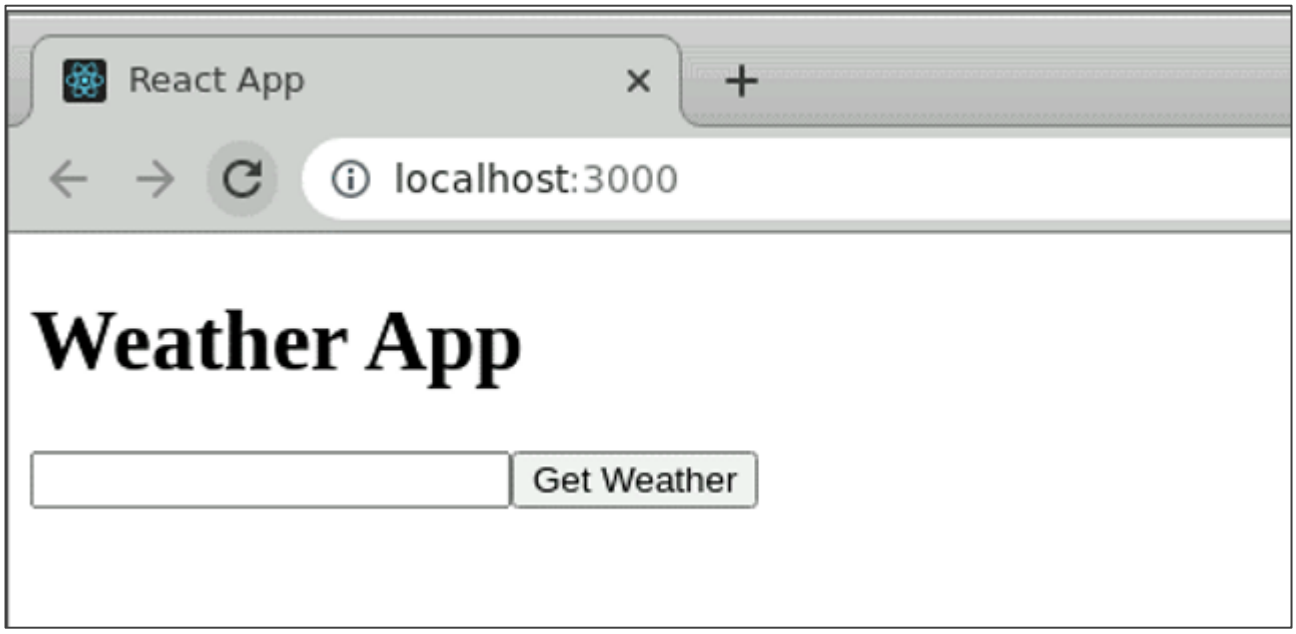
Step 11: Run the app and view it in the browser

11.1 In the terminal, run the given command to start the development server:
npm start



11.2 Open your browser and navigate to **http://localhost:3000**

The app should be running, and you should see a simple weather app where you can enter a city and get the current weather data displayed.



By following these steps, you have successfully built a React application that demonstrates data retrieval using Redux. It enables users to input a city to fetch and display the current weather data. The application adheres to best practices, organizing concerns into separate files and components for improved maintainability and reusability.